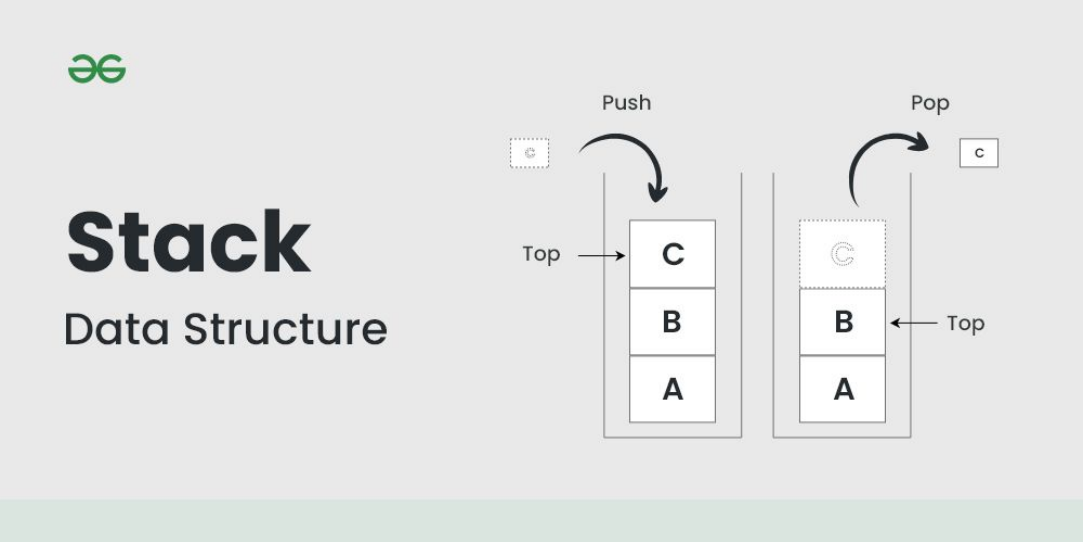
Estructuras de Datos

2026
Primer Cuatrimestre

REPASO

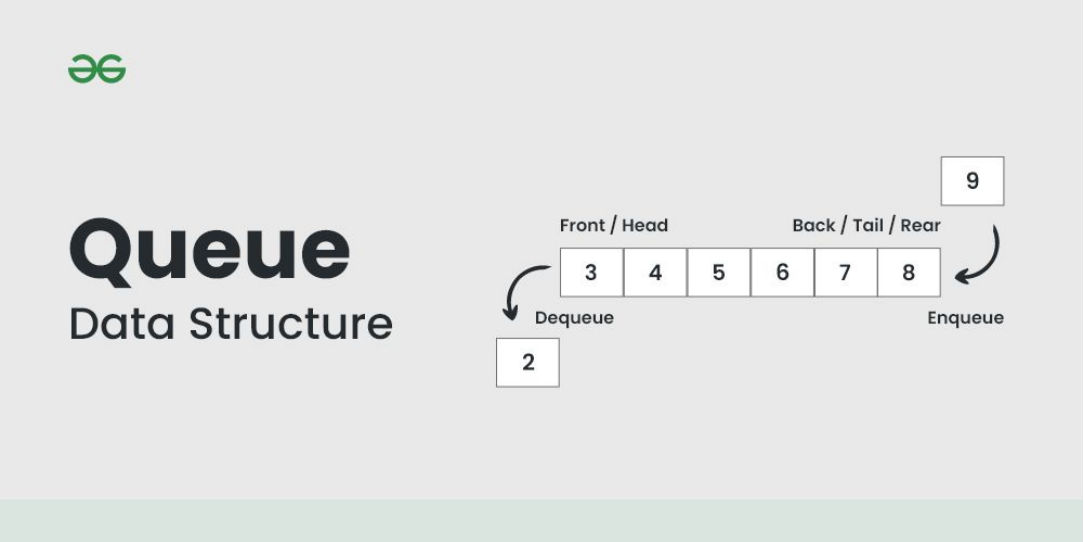
TDA Pila / ADT Stack



Si alguien necesita **un elemento**, lo saca del tope de la pila →***pop***

Cuando alguien **agrega un elemento** lo pone en el tope de la pila → ***push***

TDA Cola / ADT Queue



enqueue →agrega un elemento al final

dequeue → elimina un elemento del frente y lo retorna

Clase genérica vs método genérico

```
public class Libreria<E> {
        public boolean contiene(Stack<E> pila, E
item) {
                boolean encontre = false;
                while ( !pila.isEmpty() && !encontre ) {
                        if ( pila.top().equals(item) )
                                encontre = true;
                        else
                                pila.pop();
                }
                return encontre;
        }
}
```

```
public class Aplicacion {
  public static void main(String [] args) {
        Libreria<Character> lib = new Libreria<>();
        Stack<Character> pila = new  PilaConArreglo <Character>();
        //..... cargar una pila de chars con algo....
        boolean estaLaA = lib.contiene(pila, 'a');

        Libreria<Integer> lib = new Libreria<>();
        Stack<Integer> pilaInt = new PilaConArreglo<Integer>();
        //..... cargar una pila de ints con algo....
        boolean estaEl8 = lib.contiene(pilaInt, 8);
  }
}
```

Clase genérica vs método genérico

```
public class Libreria {
        public <E> boolean contiene(Stack<E> pila,
E item) {
                boolean encontre = false;
                while ( !pila.isEmpty() && !encontre ) {
                        if ( pila.top().equals(item) )
                                encontre = true;
                        else
                                pila.pop();
                }
                return encontre;
        }
}
```

```
public class Aplicacion {
 public static void main(String [] args) {
        Libreria lib = new Libreria();
        Stack<Character> pila = new  PilaConArreglo <Character>();
        //..... cargar una pila de chars con algo....
        boolean estaLaA = lib.contiene(pila, 'a');

        Stack<Integer> pilaInt = new PilaConArreglo<Integer>();
        //..... cargar una pila de ints con algo....
        boolean estaEl8 = lib.contiene(pilaInt, 8);
  }
}
```

Estructuras Enlazadas

Queriendo hacer el avión más grande



Queriendo hacer el avión más grande



Queriendo hacer el tren más grande





Estructuras Enlazadas

La facilidad con la que yo puedo modificar una cosa va a depender de la flexibilidad de la unión de las partes que conforman la cosa

Estructuras Enlazadas

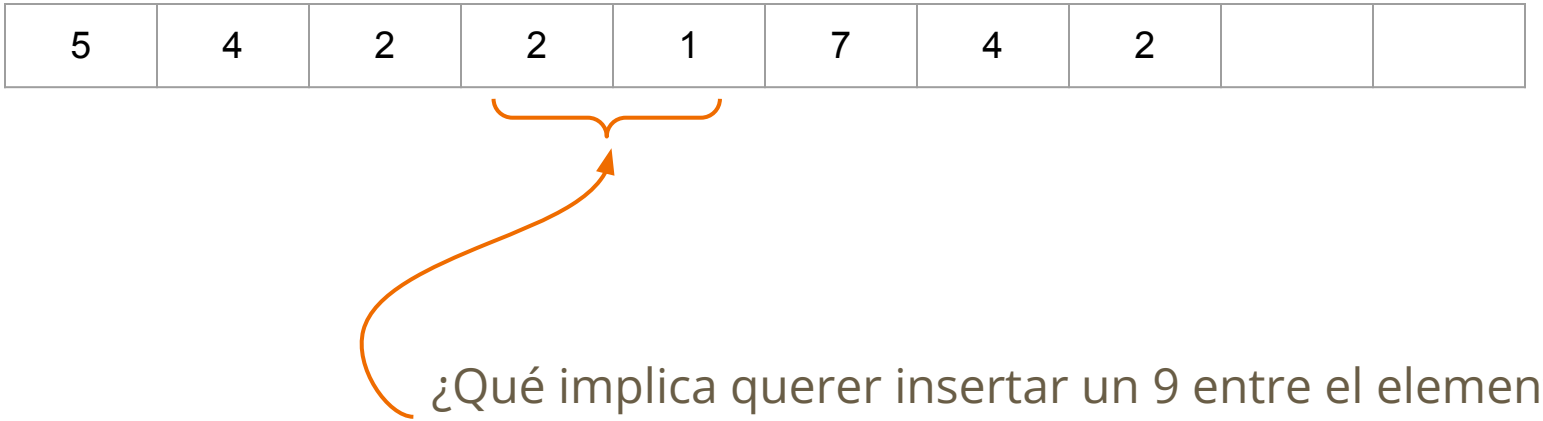
La facilidad con la que yo puedo modificar una cosa va a depender de la flexibilidad de la unión de las partes que conforman la cosa

La facilidad con la que yo puedo modificar una estructura va a depender de la flexibilidad de la unión de los elementos que conforman la estructura

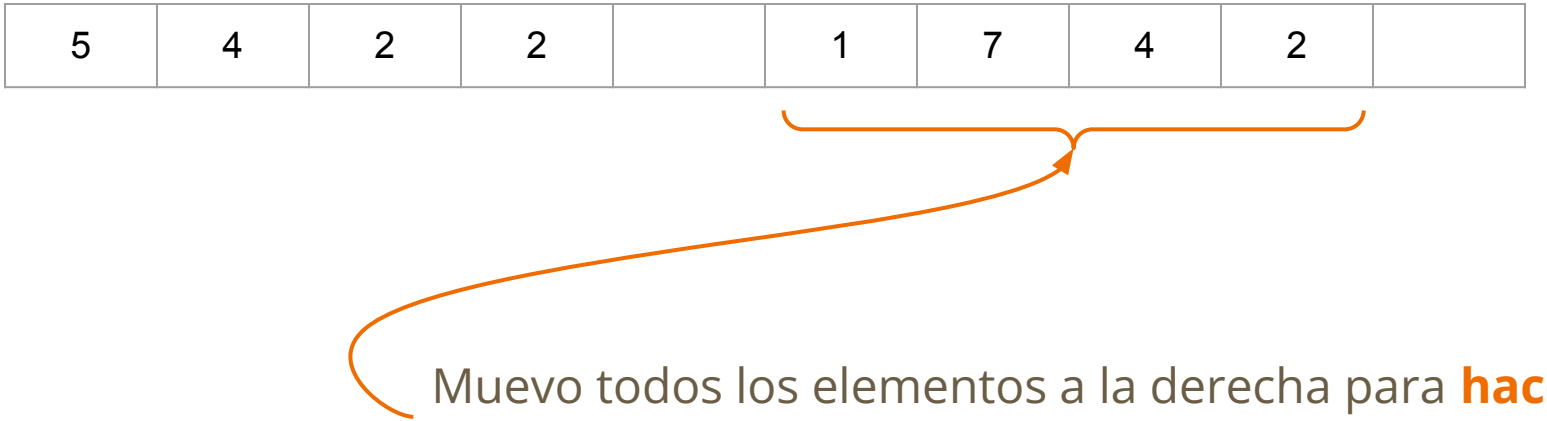
Estructura no Enlazada / Arreglo

5	4	2	2	1	7	4	2		

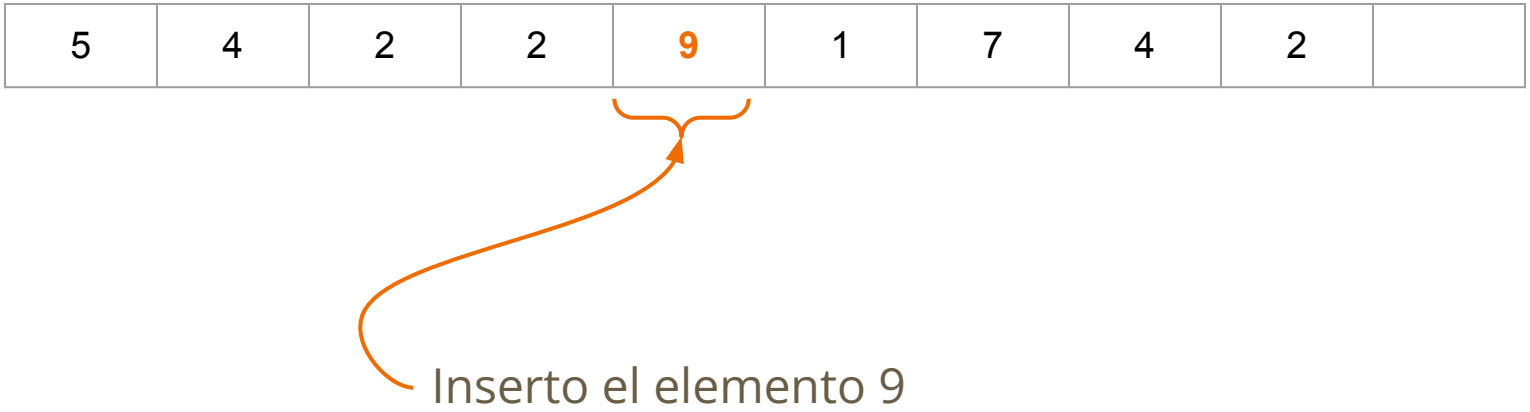
Estructura no Enlazada / Arreglo



Estructura no Enlazada / Arreglo



Estructura no Enlazada / Arreglo



Estructura no Enlazada / Arreglo

El Arreglo, como estructura de datos, no es muy flexible cuando se trata de hacer lugar en el medio para poner algo.

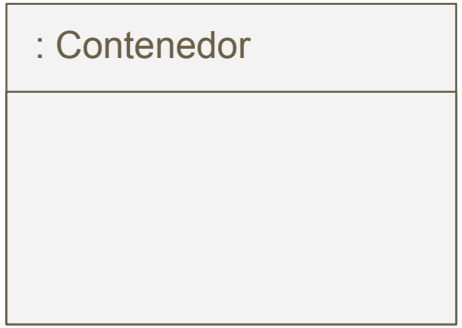
5	4	2	2	9	1	7	4	2	

Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (se conecta como los vagones de un tren) con otra caja y así sucesivamente

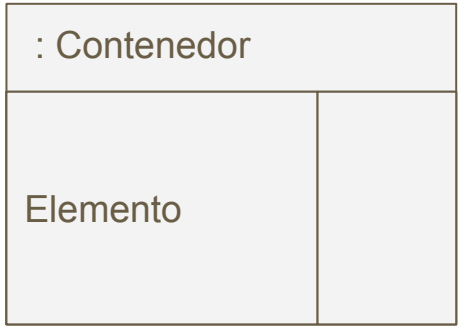
Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (conecta, toma de la mano) con otra caja y así sucesivamente



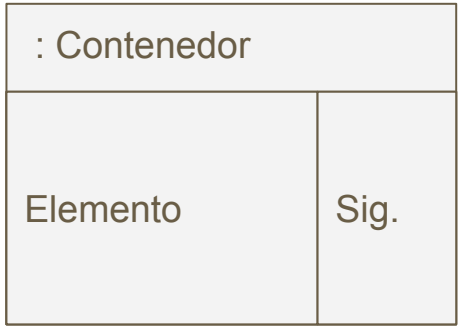
Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (conecta, toma de la mano) con otra caja y así sucesivamente



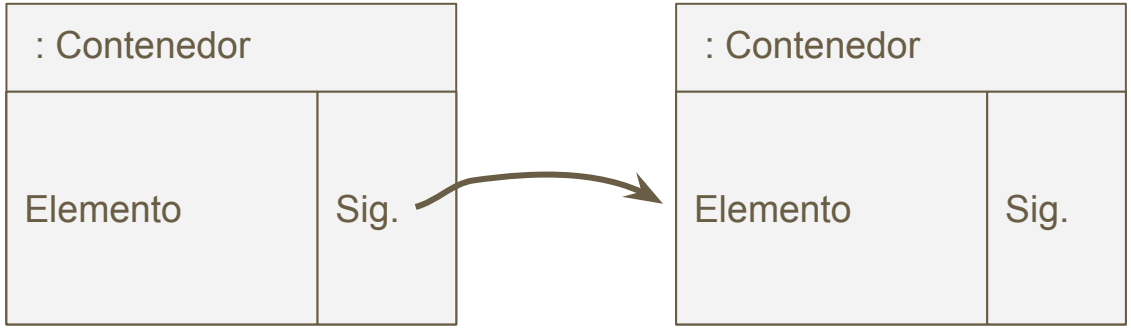
Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (conecta, toma de la mano) con otra caja y así sucesivamente



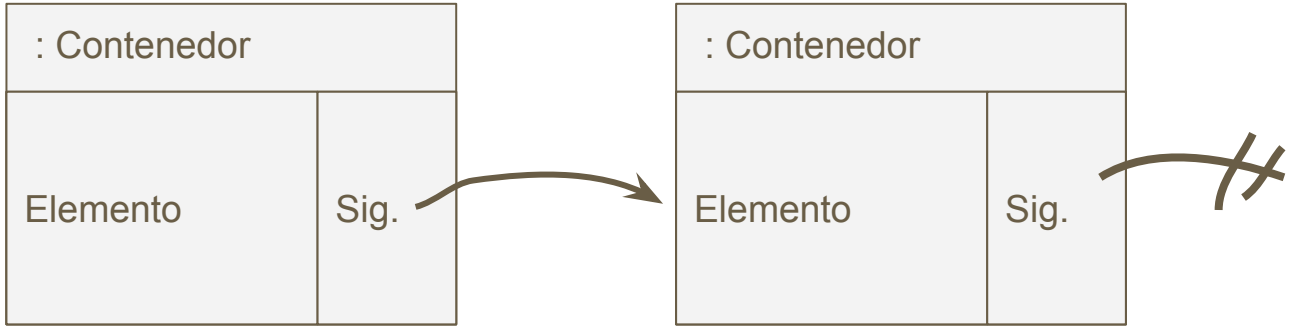
Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (conecta, toma de la mano) con otra caja y así sucesivamente

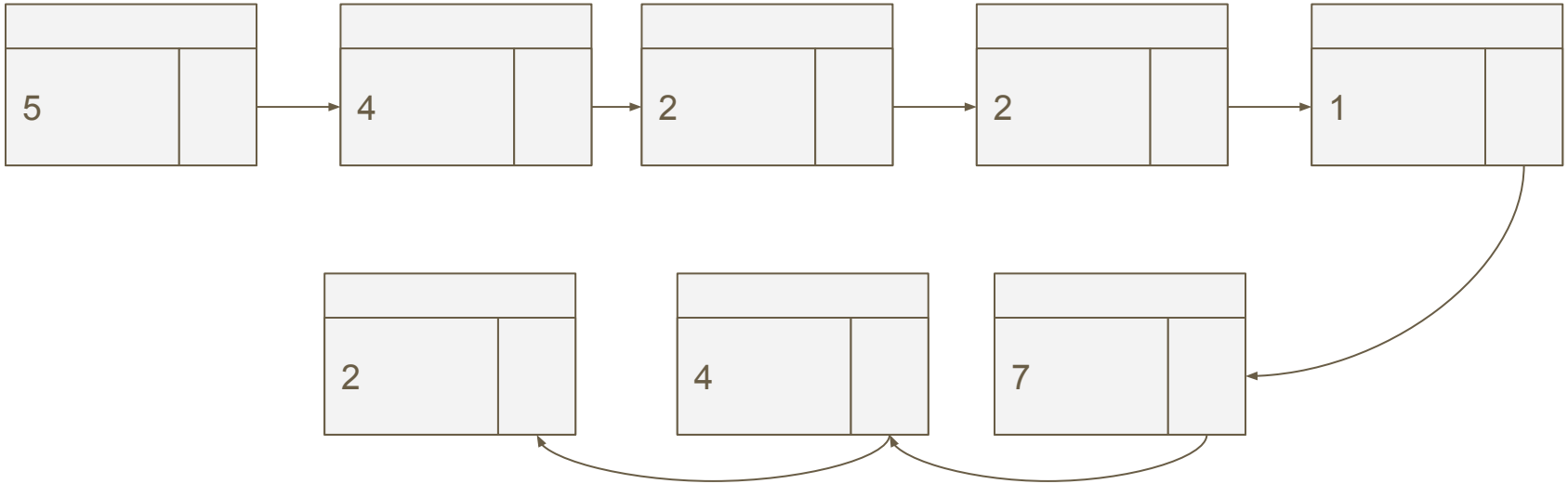


Estructuras Enlazadas

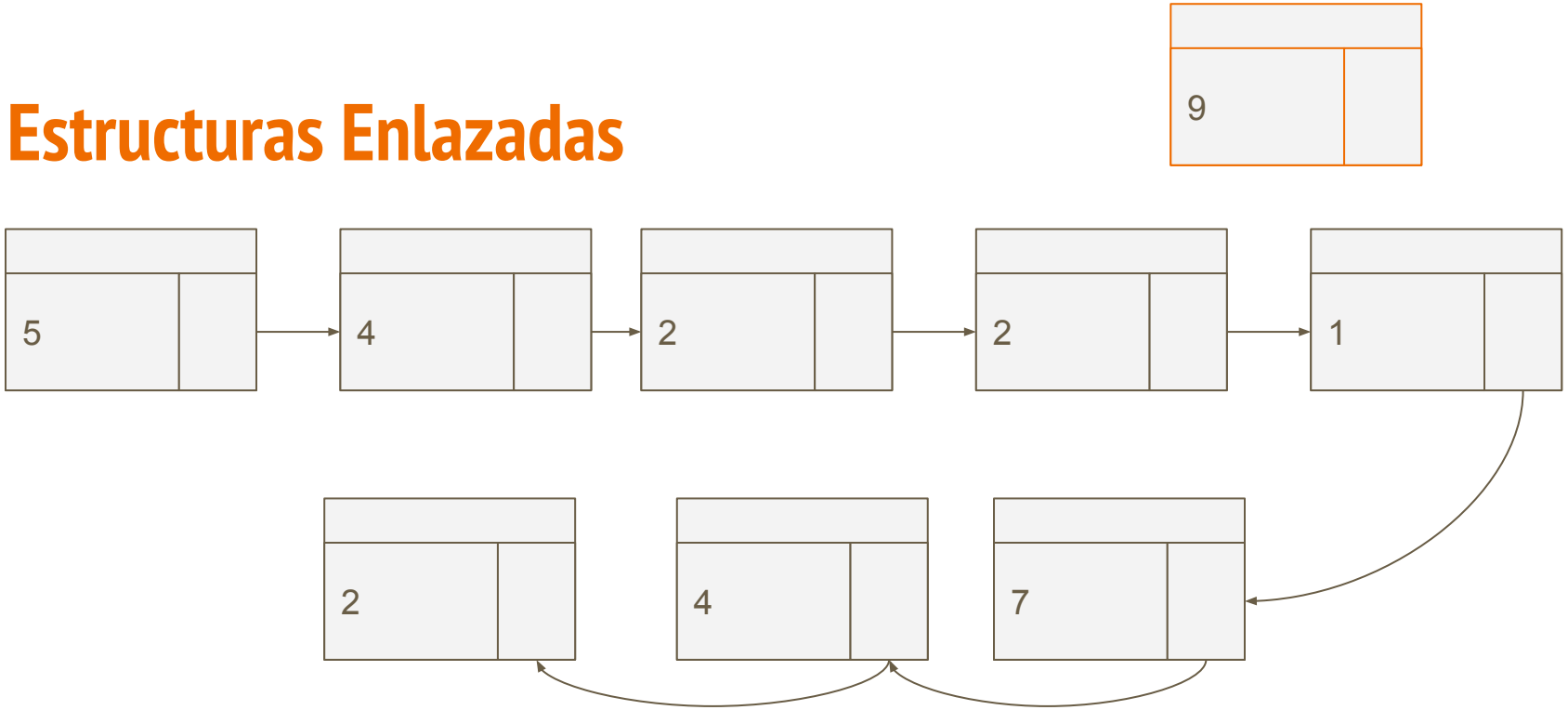
Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (conecta, toma de la mano) con otra caja y así sucesivamente



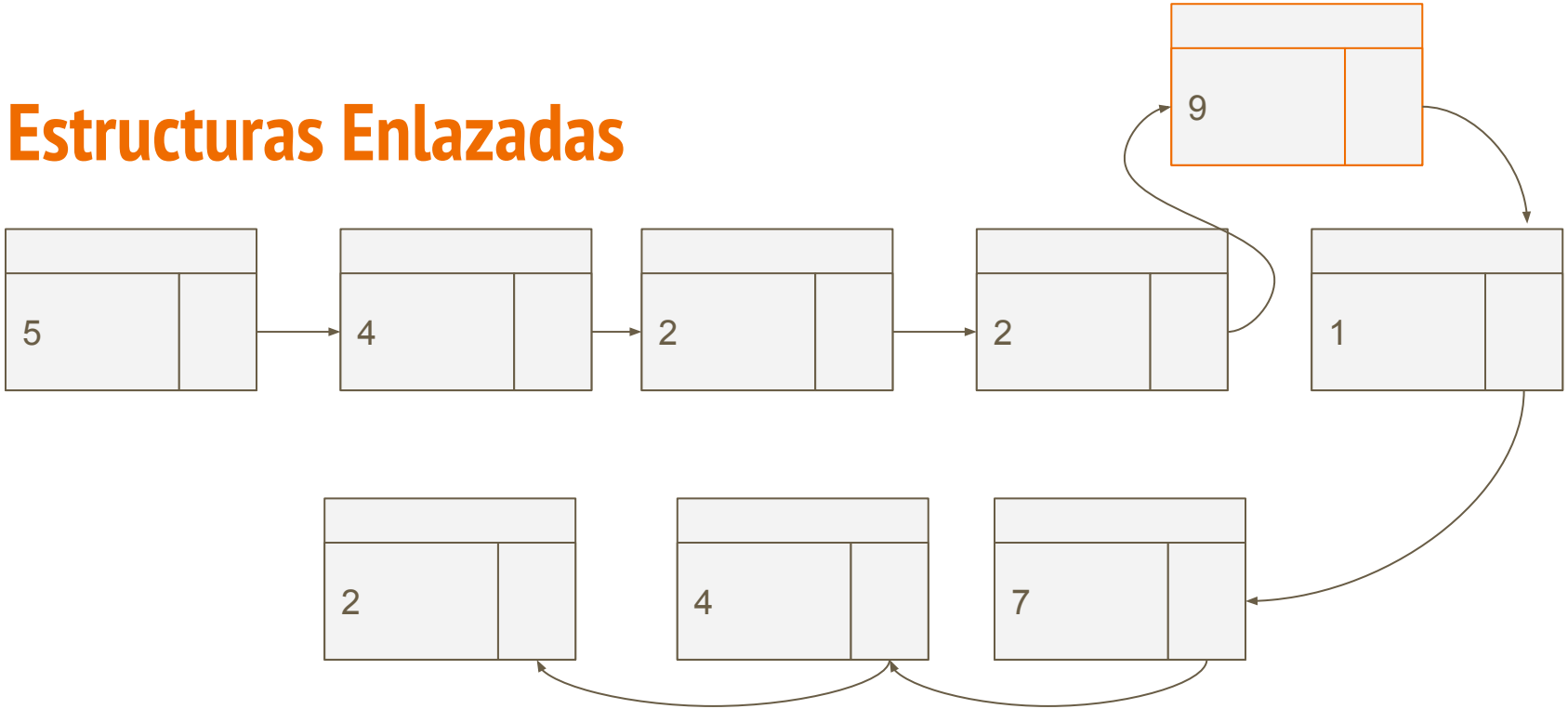
Estructuras Enlazadas



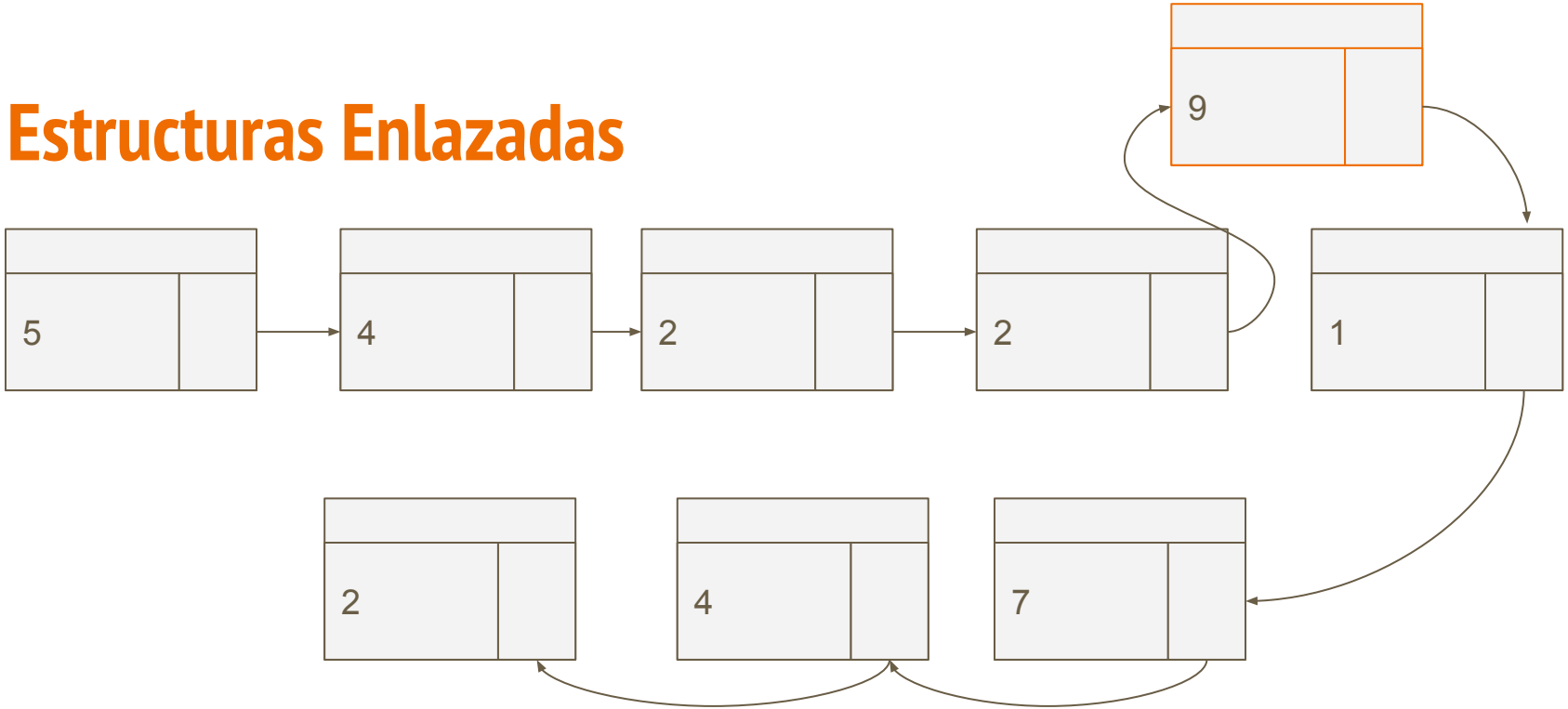
Estructuras Enlazadas



Estructuras Enlazadas



Estructuras Enlazadas



A diferencia de lo que pasó con el Arreglo, agregar un nuevo elemento acá no me implica ningún movimiento en el resto de los elementos

Node<E>

Node<E>

Node<E>

```
public class Node<E> {
        protected E element;
        protected Node<E> next;
        public Node( E e ) {...}
        public E getElement() {...}
        public void setElement( E element ) {...}
        public Node<E> getNext() {...}
        public void setNext( Node<E> e ) {...}
}
```

Recursividad en la definición de tipos

El constructor podría definirse con distintos argumentos (recibir element, next, nada)

TDA Pila con Nodo<E>

TDA Pila con Nodo<E>

TDA Pila con Nodo<E>

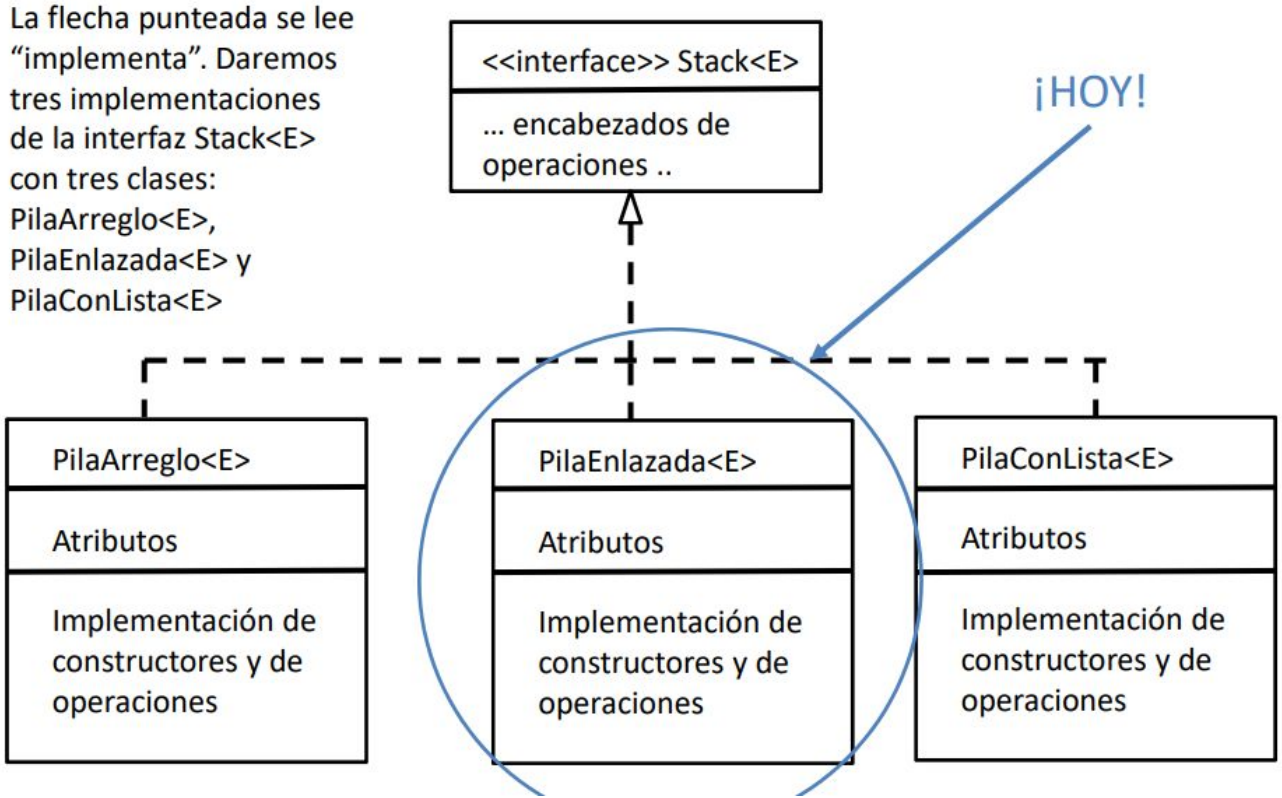
Hasta ahora hicimos/discutimos/implementamos

TDA Pila con arreglos[]

(+) Fácil de implementar, un array y un índice

(-) El array nos limita la capacidad de la estructura

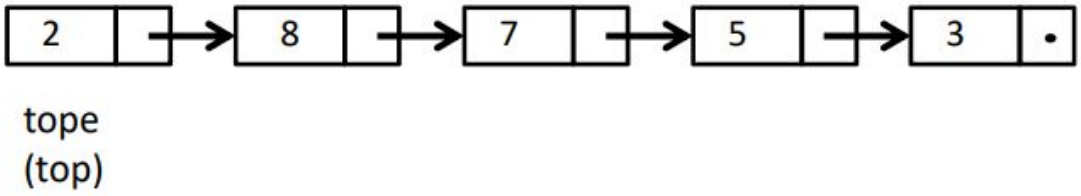
Diagrama UML de diseño



TDA Pila con nodos enlazados

- Utilizaremos una estructura de nodos enlazados
- La pila mostrada se representará con nodos (celdas) enlazados:
  - La pila conoce sólo el nodo cabeza/tope, que corresponde al tope o último dato apilado
  - Todas las operaciones son eficientes pues la estructura es no acotada y no hay casos especiales porque nunca se llena
  - Es más difícil de codificar, depurar y mantener pero se gana en flexibilidad
  - Se puede almacenar el tamaño como un atributo más para evitar recorrer la estructura para computar size().

2
8
7
5
3



TDA Pila con nodos enlazados: main cliente

```
// Archivo: PilaEnlazada.java
public class PilaEnlazada<E> implements Stack<E> {
...
}
```

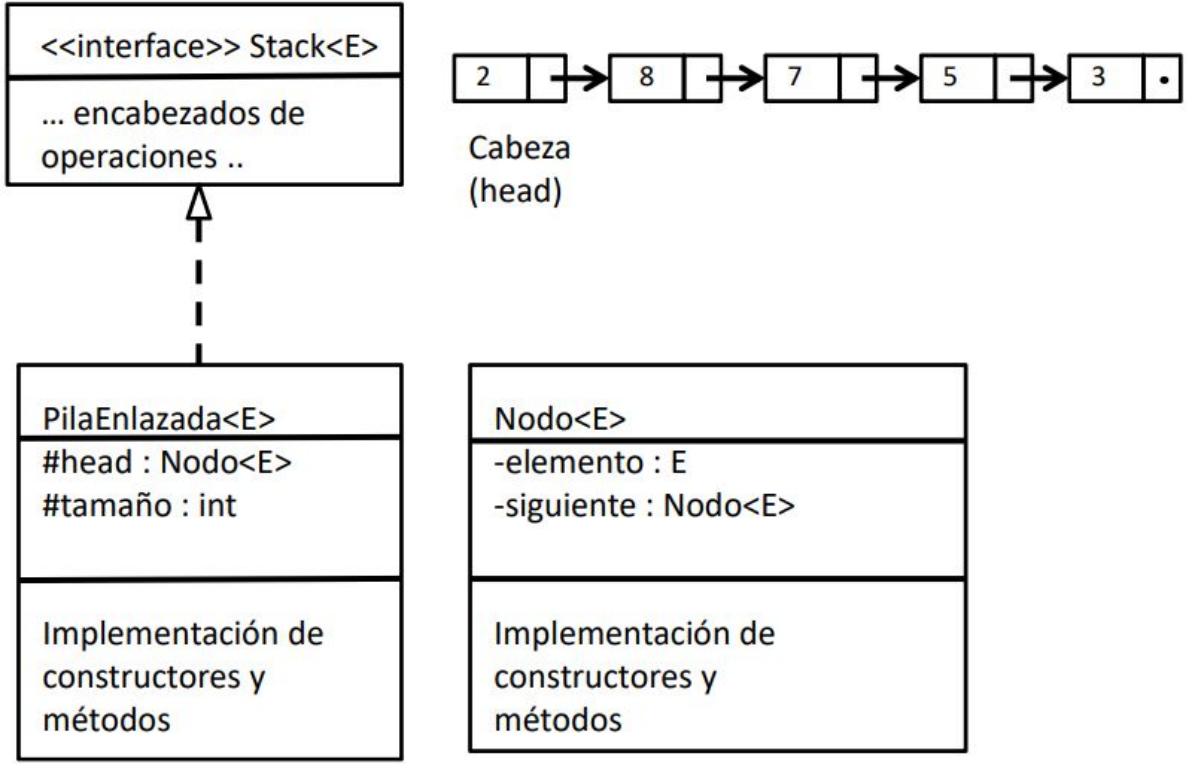
```
// Archivo: AplicacionUsaPilaEnlazada.java
public class AplicacionUsaPilaEnlazada {
    public static void main( String [] args ) {
            Stack<Integer> s = new PilaEnlazada<Integer>();
            s.push( 3 );
            s.push( 5);
            s.push( 7 );
            s.push( 8 );
            s.push( 2 );
    }
}
```

Ahora la aplicación debe invocar el constructor de la nueva clase pero como la variable s es de tipo Stack el resto de la implementación de main no cambia.

8

2
8
7
5
3

TDA Pila con nodos enlazados: estructura



TDA Pila con nodos enlazados: constructor

Recordemos que:

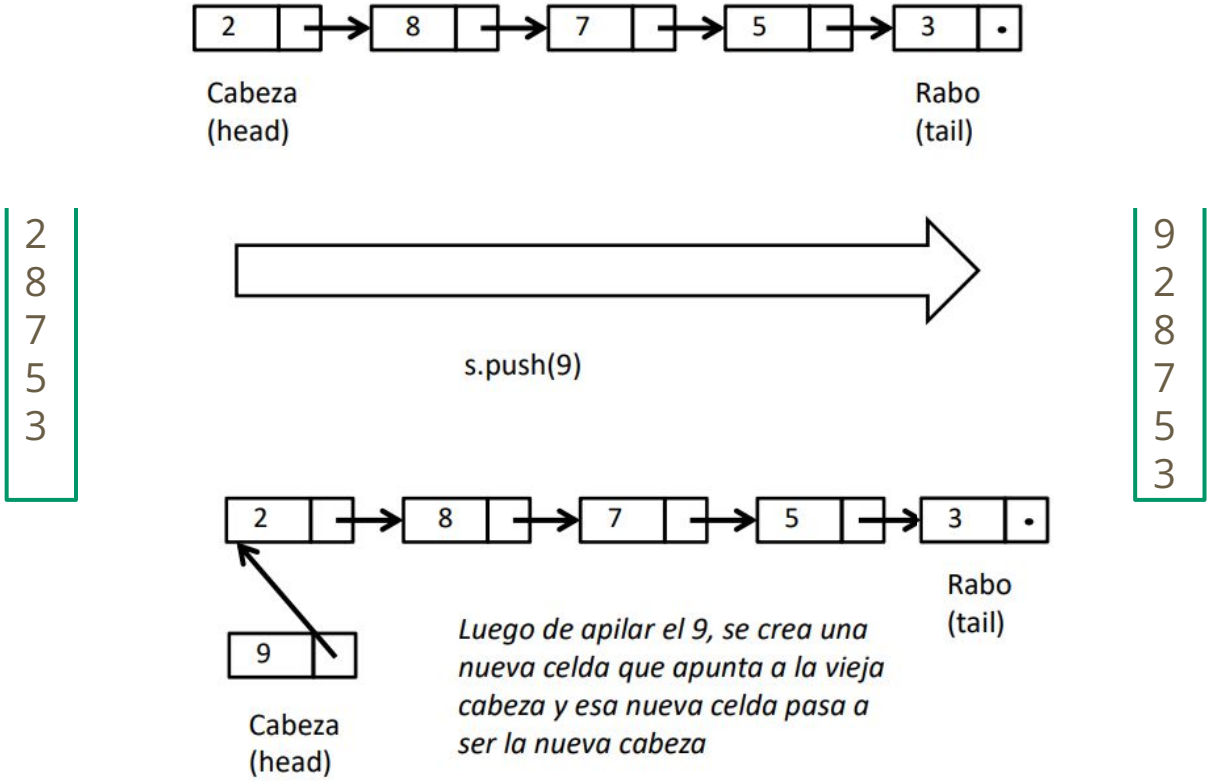
```
Public class PilaEnlazada<E>
                implements Stack<E> {
  protected Nodo<E> head;
  protected int tamaño;

  ....
}
```

Constructor:

```
PilaEnlazada() {
  head = null
  tamaño = 0
}
```

TDA Pila con nodos enlazados: apilar



TDA Pila con nodos enlazados: apilar

```
Public class PilaEnlazada<E>
                implements Stack<E> {
  protected Nodo<E> head;
  protected int tamaño;

  ....
}
```

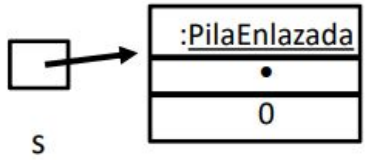
```
push(item : E)
{
Nodo<E> aux  =  new Nodo<E>(item )
aux.setElemento( item )
aux.setSiguiente( head )
head = aux
tamaño = tamaño + 1
}
```

¿Cómo sería la traza al insertar elementos?
Notar que el constructor podría simplificarse a dos líneas de código

TDA Pila con nodos enlazados: isEmpty()

```
isEmpty() : boolean
{
return head == null
}
```

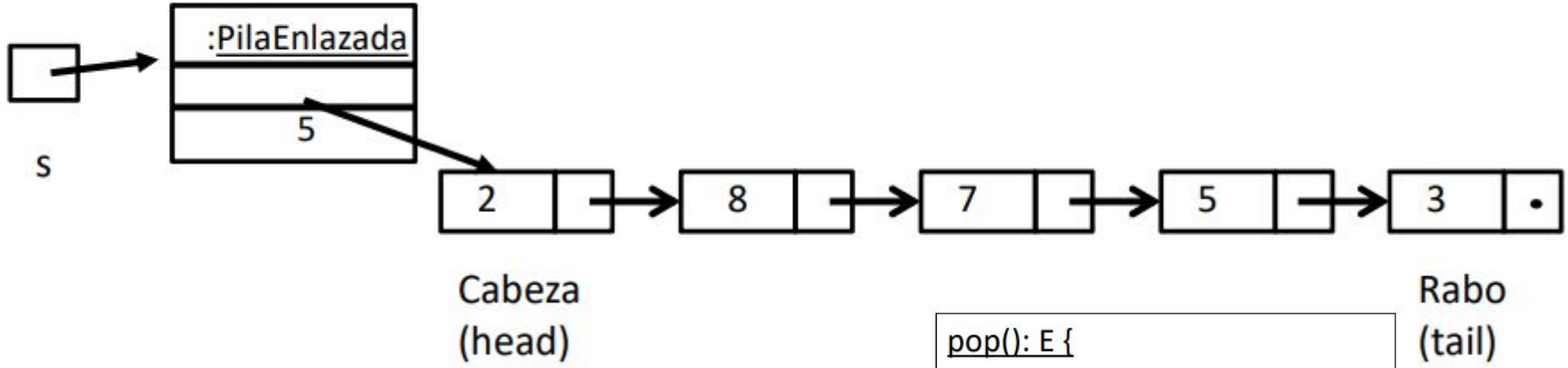
En este caso como la pila *s* está vacía el campo *head* contiene null:



En este otro caso como la pila *s* no está vacía el campo *head* apunta al nodo tope (que contiene 5):



TDA Pila con nodos enlazados: desapilar



2
8
7
5
3

Desapilar requiere eliminar la celda que contiene el 2 y hacer que la celda que contiene el 8 sea la nueva cabeza.

- Como la celda del 2 no está referenciada por nadie, eventualmente será reclamada por el recolector de basura aunque la celda del 2 siga apuntando a la celda del 8.
- Cuando la pila tiene un único elemento, el código tiene como efecto que head termine con valor null.

```
pop(): E {

Si isEmpty() entonces
    error( "Pila vacia" )
Sino
    aux = head.getElemento()
    head = head.getSiguiente()
    tamaño = tamaño – 1
    retornar aux
}
```

TDA Pila con nodos enlazados:

¿Por qué conviene insertar en la cabeza de la lista y no al final?

Hacer una inserción o remoción implica recorrer toda la lista
Ineficiente

TDA Cola con Nodo<E>

TDA Cola con Nodo<E>

Hasta ahora hicimos/discutimos/implementamos

TDA Cola con arreglos (circulares) []

(-) Requiere aritmética de módulo para funcionar eficientemente

(-) El array nos limita la capacidad de la estructura

TDA Cola con Nodo<E>

¿Qué problema tiene esta idea?

```
9
0  public class MiQueueNode<E> implements Queue<E> {
1      protected Node<E> frente;
2      protected int cantidad;
3
```

TDA Cola con Nodo<E>

El problema con esta estructura es que, si queremos hacer las cosas fáciles, necesitamos un acceso fácil a los dos extremos de la estructura

TDA Cola con Nodo<E>

¿Qué problema tiene esta idea?

```
9
0 public class MiQueueNode<E> implements Queue<E> {
1     protected Node<E> frente;
2     protected int cantidad;
3
```

Tenemos que ir a buscar el otro extremo cada vez que se inserta un elemento

TDA Cola con Nodo<E>

¿Qué problema tiene esta idea?

```
9
0 public class
1     protecte
2     protecte
3
```



Tenemos que ir a buscar el otro extremo cada vez que se inserta un elemento

TDA Cola con Nodo<E>

```
public class MiQueueNode<E> implements Queue<E> {
    protected Node<E> frente;
    protected Node<E> fondo;
    protected int cantidad;

    public MiQueueNode() {
        this.frente = null;
        this.fondo = null;
        this.cantidad = 0;
    }

    @Override
    public boolean isEmpty() {
        return this.cantidad == 0;
    }
```

¿Cual debería ser el lugar de insertado?

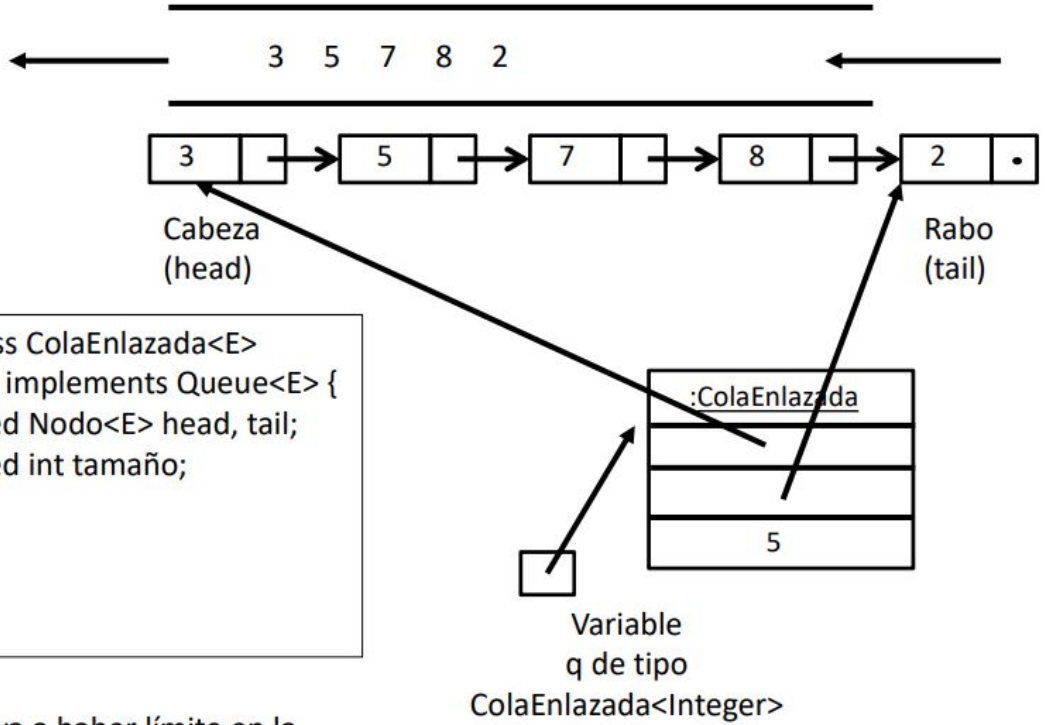
TDA Cola con Nodo<E>

```
public class MiQueueNode<E> implements Queue<E> {
    protected Node<E> frente; // por acá saco
    protected Node<E> fondo; // por acá inserto
    protected int cantidad;

    public MiQueueNode() {
        this.frente = null;
        this.fondo = null;
        this.cantidad = 0;
    }

    @Override
    public boolean isEmpty() {
        return this.cantidad == 0;
    }
```

Implementaciones de colas con nodos enlazados

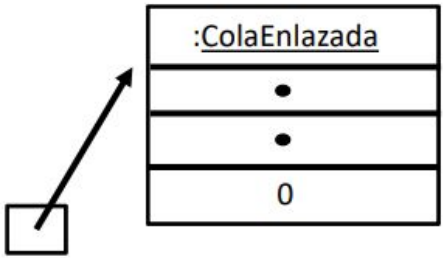



```
public class ColaEnlazada<E>
            implements Queue<E> {
  protected Nodo<E> head, tail;
  protected int tamaño;

  ....
}
```

Nota: No va a haber límite en la cantidad de elementos que se pueden encolar.

TDA Cola con nodos enlazados: constructor




```
Queue<Integer> q = new ColaEnlazada<Integer>();
```

```
public class ColaEnlazada<E>
            implements Queue<E> {
  protected Nodo<E> head, tail;
  protected int tamaño;

  ....
}
```

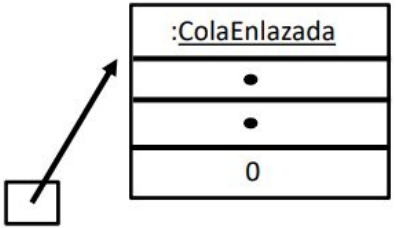
```
public ColaEnlazada() {
    head = null
    tail = null;
    tamaño = 0;
}
```

Estrategias para encolar (enqueue) y desencolar (dequeue)

1. Plantear un caso cuando la estructura está vacía
2. Plantear otro caso cuando la estructura tiene un único nodo
3. Plantar otro caso cuando la estructura tiene 2 nodos o más

- Como tenemos 3 alternativas, la estructura de control a elegir será un if-then-else anidado
- Refactorizar:
  - si hay casos que son iguales
  - si hay sentencias comunes al principio o al final de los if-else
- En el caso de esta teoría mostramos la implementación de libro que ya tiene en cuenta estas cuestiones

TDA Cola con nodos enlazados: encolar (cola vacía)



q

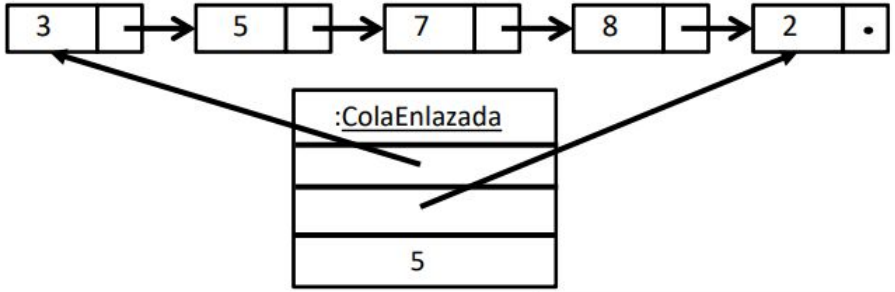
Quiero ejecutar:

q.enqueue(3);

Veamos la traza:

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if (head == null )
5.          head = nodo;
6.  else
7.          tail.setSiguiente( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

TDA Cola con nodos enlazados: encolar (cola no vacía)



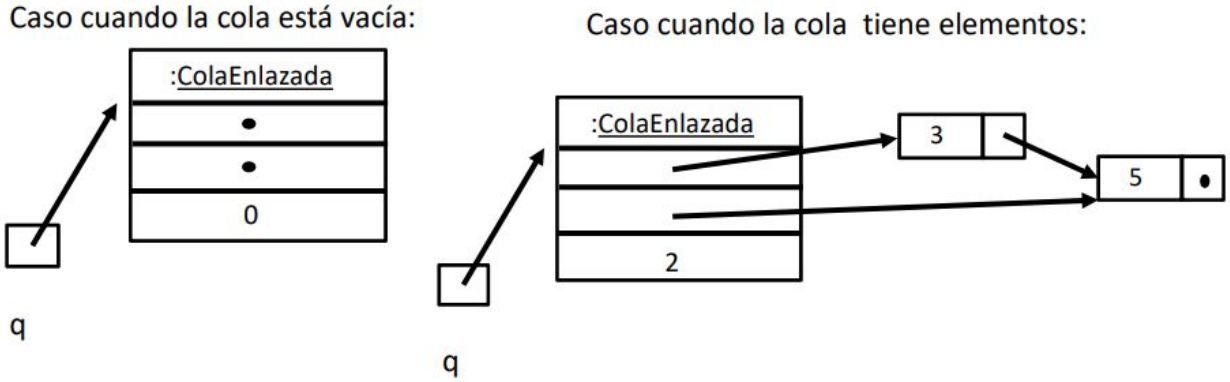
Quiero ejecutar

q.enqueue(4);

Veamos la traza

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.          head = nodo;
6.  else
7.          tail.setSiguiente( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

TDA Cola con nodos enlazados: isEmpty()

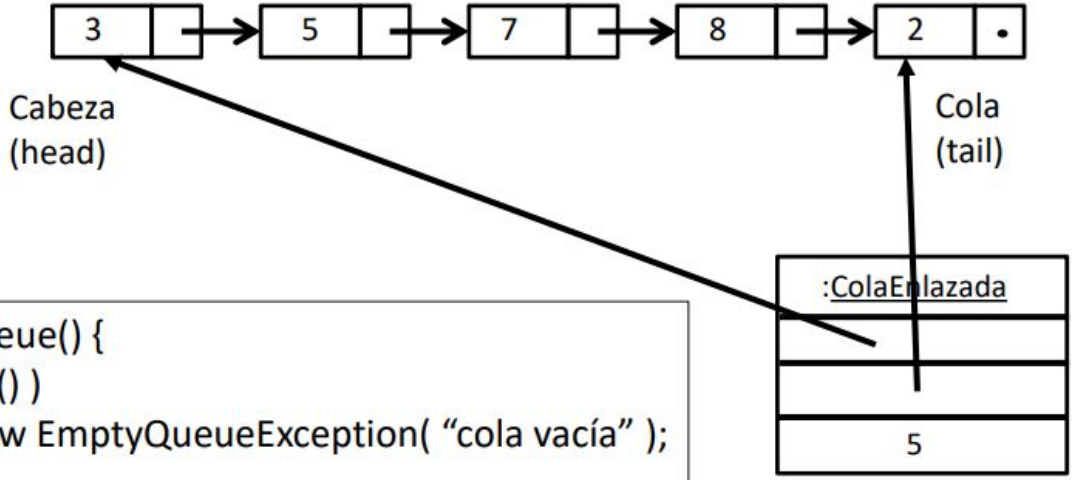


isEmpty() devuelve un booleano que se puede obtener de una de 2 formas:

1. Testear si tamaño es 0

2. Testear si head es null

TDA Cola con nodos enlazados: desencolar




```
public E dequeue() {
1.  if( isEmpty() )
2.     throw new EmptyQueueException( “cola vacía” );

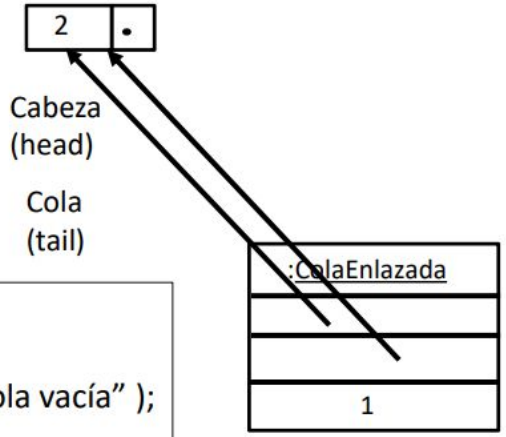
3.  E tmp = head.getElemento();
4.  head = head.getNext();
5.  tamaño --;
6.  if( head == null ) tail = null;
7.  return tmp;
}
```

Quiero ejecutar

q.dequeue();

Veamos la traza.

TDA Cola con nodos enlazados: desencolar (q.size ==1)




```
public E dequeue() {
1.  if( isEmpty() )
2.     throw new EmptyQueueException( “cola vacía” );

3.  E tmp = head.getElemento();
4.  head = head.getNext();
5.  tamaño --;
6.  if( head == null ) tail = null;
7.  return tmp;
}
```

Ahora head y tail apuntan al mismo nodo.

THE END

Bibliografía

• Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5

• Se puede profundizar la parte de nodos enlazados en la Sección 3.2 del libro